Geo-*metric* Learning: Deep Metric Learning for Numerical Geometry

Gautam Pai and Ron Kimmel

Abstract Metric learning schemes are a set of techniques in machine learning that learn representations or features from data by satisfying some notion of predefined distance. Instead of designing them axiomatically, metric learning schemes enable the learning of such features from training examples, and have proven to be very useful for the purposes of semantic recognition and classification. In this chapter we revisit metric learning from a different perspective. In particular, we explore their application to problems of numerical geometry. We consider representations that exhibit a strong geometric structure and try to learn them from examples. Specifically, we explore two problems that use artificial neural networks in a metric learning setup. First, we show that we can learn the curvature, or a related local invariant structure, of a planar curve and second, develop a learning scheme to generalize multidimensional scaling for flattening high-dimensional manifolds. We show that learned geometric representations demonstrate better robustness to noise and improved generalization to unseen data points.

1 Introduction

Geometric analysis is an important aspect of a large number of problems in computer vision, computer graphics and image processing. It is essential to develop strong insight and intuition of the geometry involved in problems that deal with attributes like 3D structure of scenes, parametric surfaces, boundaries of object segments, etc. The objective here is to discover and develop representations which are invariant to the prevailing physical and geometric constraints of the problem. For example,

Gautam Pai
Faculty of Computer Science, Technion, e-mail: paigautam@cs.technion.ac.il

Ron Kimmel
Faculty of Computer Science, Technion, e-mail: ron@cs.technion.ac.il

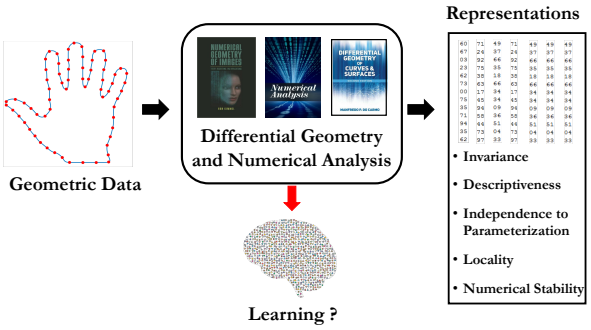

Fig. 1 Schematic for combining geometry with learning. The goal is to replace components of differential geometry and numerical analysis, with a learnable computation, and (re)discover essential geometric properties from the data.

consider the problem of depth estimation of a scene from images captured from different viewpoints. It involves finding features from the image which are invariant to the camera geometry and other artifacts like occlusion and noise. Similarly, the problem of correspondence and registration of curves or surfaces involve finding geometric representations which are invariant to rotations, translations, scale, skew, articulations and isometric deformations. It is essential for such representations to not only be invariant to the set of transformations but also to the parametrization of the data in order to be universal in their usage. Typically this is done by carefully modeling the problem with tools from differential geometry, differential equations and systems theory and then using numerical techniques for engineering them to the problem at hand. A comprehensive treatise on problems of numerical geometry can be found in [23, 7].

Learning methods involve the study and design of algorithms that develop functional architectures which can be trained to perform a specific task when they are endowed with an objective cost function and a sufficient amount of data. The entire process of modeling the problem and engineering it to a specific application is subsumed in the learning process. Recent advances in convolutional neural networks and deep learning systems have catapulted their usage in solving the multitude of problems in computer vision, speech, pattern recognition and many other data-sciences. Their success has been largely attributed to the meaningful semantic features they generate from the data which enables them to perform tasks like recognition, classification, discrimination etc. with superiority.

We propose to connect these two fields by developing learning architectures which can extract meaningful representations from geometric data. Our focus would be to develop learning mechanisms for solving geometric problems by focusing on key attributes of invariance to both transformations and parameterization. We aim to train learning architectures which have much better numerical properties as compared to their axiomatic counterparts. Instead of designing geometric formulas by hand and

developing numerical methods to engineer them robustly to data, we explore the means to *learn* the geometric invariants from data.

In what follows, we apply a specific class of machine learning algorithms called *metric learning* schemes to learn these geometric features. Metric learning, as the name implies, are a set of techniques that learn features from data which satisfy some notion of predefined distance, all in an unsupervised manner. The typical input to a metric learning scheme is a pair of data-points and the expected distance relationship between them. This relationship could be expressed in binary terms as *close* and *far* [6, 12, 19] or possibly something more sophisticated like the actual distance between pairs of points [26]. The goal of the metric learning scheme is to discover features for each data-point, such that this expected distance relationship is preserved. By training with a large number of examples, the metric learning scheme is expected to converge to consistent features that characterize the data in terms of these expected pairwise distances.

We show two specific examples of metric learning that leads to useful geometric feature learning from data. First, in Section 2 we design a setup to *learn* the curvature (specifically an invariant representation) of a curve from examples. A first course on differential geometry shows that the arc-length and curvature are fundamental invariants of a curve and provably invariant to specific class of transformations [1, 10, 9, 38]. Hence, the Euclidean arc-length and curvature are the simplest invariants to Euclidean transformations, Affine arc-length and affine curvature to affine transformations [35]m and so on. We demonstrate experimentally that, by synthetically generating examples of these transformations, we can train a neural network to *learn* these invariants [27]. Moreover, we find them to be much more robust to noise and sampling in comparison to their axiomatic counterparts.

In the second example in Section 3, we devise a parametric scheme to enable multidimensional scaling [26]. Multidimensional scaling are a set of methods that output a configuration for data points such that input pairwise distances are preserved. Over the years, the scheme of preserving distances has become sophisticated by different techniques that perform flattening using only a sparse input of distances. We demonstrate a metric learning setup, where a deep network is shown to perform high quality flattening of high dimensional manifolds with only a few input pairwise distances. Moreover, the parametric nature of the learning, ensures a trivial out-of-sample extension and more importantly, brings a geometric perspective to the generalizability of deep learning.

2 Learning Geometric Invariants of Planar Curves

2.1 Background

Invariant representations or *invariants* are functions defined over geometric structures which do not change despite the action of a certain type of transformation. According to Kendall [22], shape is what remains after translation, rotation and

scale have been factored out. In general invariant theory, this can be extended to any general Lie group transformation. Depending on our modeling of the problem we can state that shape is what remains after the effects of a certain type of deformation is factored out. These invariants provide a means to intrinsically characterize the geometric structure and hence, their efficient and robust computation is essential to the success of a given geometric processing framework.

A powerful theorem from E. Cartan [11] and Sophus Lie [1] characterizes the space of these invariant signatures. It begins with the concept of arc-length which is a generalized notion of the length along a curve. Given a type of transformation, one can construct an intrinsic arc-length that is independent of the parameterization of the curve, and compute the curvature with respect to this arc-length. The fundamental invariants of the curve, known as differential invariants [9], [10] are the set of functions comprising of the curvature and its successive derivatives with respect to the invariant arc-length. These differential invariants are unique in a sense that two curves are related by the group transformation if and only if their differential invariant signatures are identical. Moreover, every invariant of the curve is a function of these fundamental differential invariants.

Consider $C(p) = \begin{bmatrix} x(p) \\ y(p) \end{bmatrix}$, a planar curve with coordinates x and y parameterized by some parameter p. The Euclidean arc-length, is given by

$$s(p) = \int_0^p |C_p| \, dp = \int_0^p \sqrt{x_p^2 + y_p^2} \, dp, \tag{1}$$

where $x_p = \frac{dx}{dp}$, and $y_p = \frac{dy}{dp}$ and the principal invariant signature, that is, the Euclidean curvature is given by

$$\kappa(p) = \frac{\det(C_p, C_{pp})}{|C_p|^3} = \frac{x_p y_{pp} - y_p x_{pp}}{(x_p^2 + y_p^2)^{\frac{3}{2}}}. \tag{2}$$

Thus, we have the Euclidean differential invariant signatures given by the set $\{\kappa, \ \kappa_s, \ \kappa_{ss} \ \ldots\}$ for every point on the curve. Cartan's theorem provides an axiomatic construction of invariant signatures and the uniqueness property of the theorem guarantees their theoretical validity. Their importance is highlighted from the fact that *any* invariant is a function of the fundamental differential invariants. In fact, for a given group of transformations, a planar curve is completely determined from just two differential signatures: $\{\kappa, \kappa_s\}$ modulo the transformation [10].

The difficulty with differential invariants is their stable numerical computation. Equations 1 and 2, involve non-linear functions of derivatives of the curve and this poses serious numerical issues for their practical implementation where noise and poor sampling techniques are involved.

At the other end, Integral invariants [25, 17, 28, 21] are invariant signatures which compute integral measures along the curve using parameterized kernels $h_r(p, x) : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$

$$I_r(p) = \int_C h_r(p, x) d\mu(x). \tag{3}$$

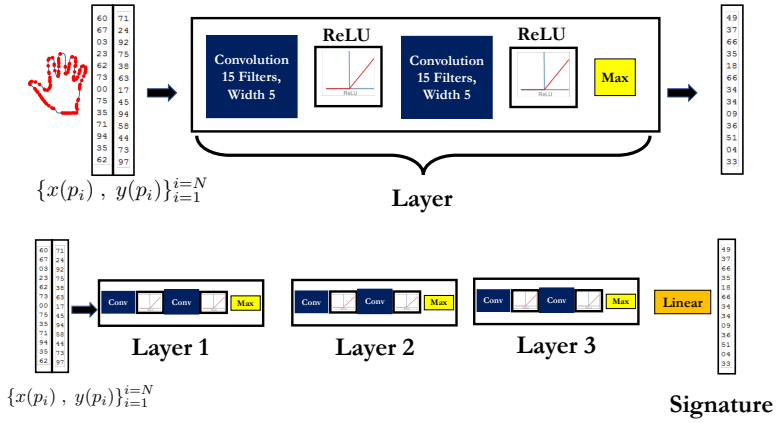

Fig. 2 A 1D convolutional neural network architecture to process discrete planar curves. The input to the architecture are the ordered 2D coordinates of a planar curve. After the sequence of convolution-ReLU operations in each layer, the architecture finally outputs a signature for each point on the curve.

For example, for each point on the contour, the integral area invariant computes the area of the region obtained from the intersection of a ball of radius r placed at that point and the interior of the contour. The integral nature of the computation gives the signature robustness to noise and by adjusting different radii of the ball r one can associate a scale-space of responses for this invariant. [17] and [28] provide a detailed treatise on different types of integral invariants and their properties. A similar approach was proposed in [37] that provides a histogram based local descriptor for shapes.

In the contrast to the principled axiomatic formulas of the differential and integral invariants, we show that we can *learn* invariants using neural networks. The formulaic structure of a numerical differentiation (high-pass filtering) or numerical integration (low pass filtering) followed by specific non-linearities (division/multiplication) can be replaced by a learnable architecture that can learn the appropriate filters to be used in conjunction to fixed non-linearities like ReLU. Figure 2 shows the example architecture used in our experiments. We observe that, the invariants learned by the network are much more stable than the corresponding axiomatic counterparts. See Figure 4.

2.2 Learning Invariant Signatures using Metric Learning

A planar curve can be represented either explicitly by sampling points on the curve or implicitly using a representation like level sets [23]. We work with an explicit representation of simple curves (open or closed) with random variable sampling of the points along the curve. Thus, every curve is a $N \times 2$ array denoting the

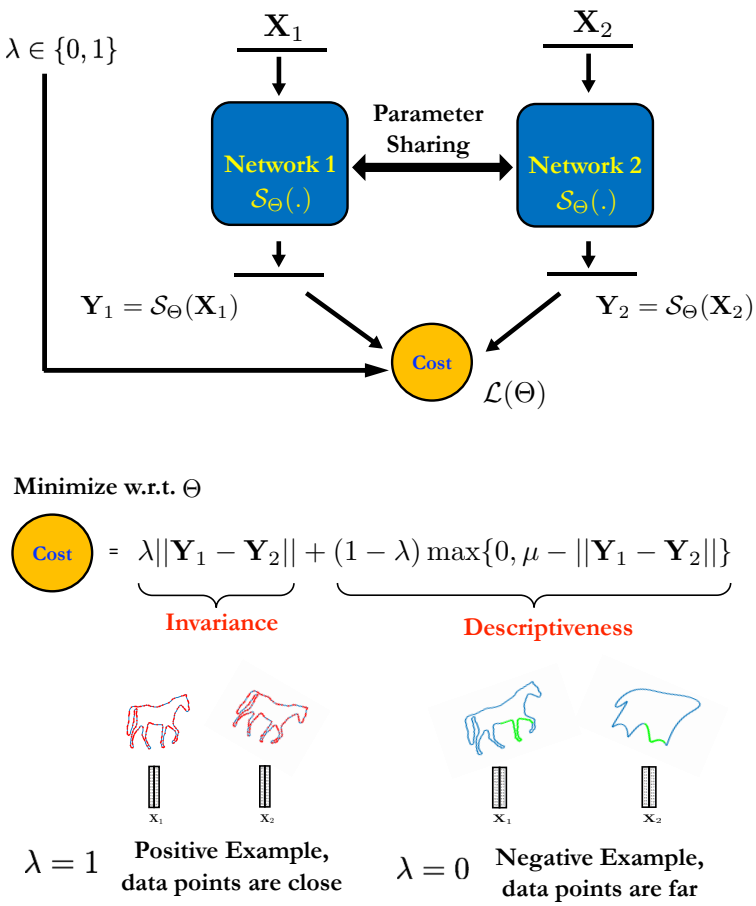

Fig. 3 *Siamese Configuration for Metric-Learning:* The inputs are pairs of data points along with the parameter λ which indicates whether the points are to be deemed similar (λ = 1) or dissimilar (λ = 0). For every pair, the network adjusts its parameters such that learned features ($\mathbf{Y_1}$, $\mathbf{Y_2}$) for similar examples become closer and features for dissimilar examples get pushed apart.

X and Y coordinates of the N points. We build a convolutional neural network which inputs a curve and outputs a representation or signature for every point on the curve. See Figure 2. The network, given by one arm of the Siamese configuration, comprises of three layers. Each layer contains two sequential batches of temporal convolutions appended with rectified linear units (ReLU) and ending with a max unit. The temporal convolution comprises of convolutions with 15 filters of width 5 as depicted in Figure 2. The max unit computes the maximum of 15 responses per point to yield an intermediate output after each layer. The final layer is followed by a linear layer which yields the final output. Since, every iteration of convolution results in a reduction of the sequence length, sufficient padding is provided on both ends of the curve. This ensures that the value of the signature at a point is the result of the

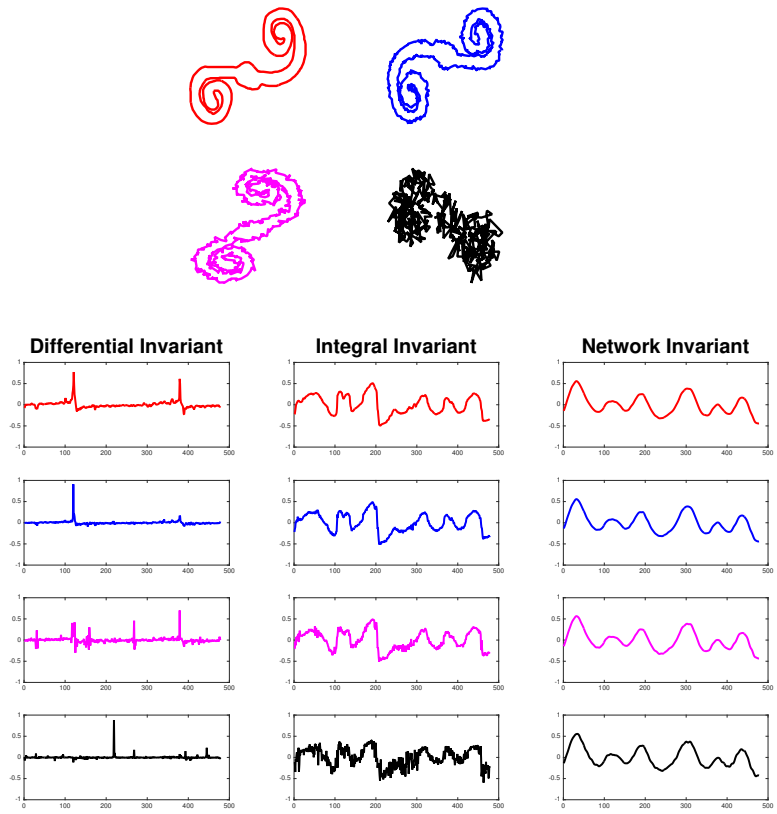

Fig. 4 Stability of different signatures in varying levels of noise and Euclidean transformations. The correspondence for the shape and the signature is the color. All signatures are normalized.

response of the computation resulting from the filter centered around that point. We can interpret this architecture as an algorithmic scheme of representing a function over the curve.

We use this architecture in a Siamese configuration which comprises of two identical networks of Figure 2 that compute signatures for two separate inputs of data. Associated to each input pair is a label which indicates whether or not that pair is a positive ($\lambda = 1$) or a negative ($\lambda = 0$) example (Figure 3). We extract geometric invariance by requiring that the two arms of the Siamese configuration minimize the distance between the outputs for the curves which are related by Euclidean transformations and maximize the distances for carefully constructed negative examples. This is enabled by the contrastive loss function [12], [24] for training the network. Let $\mathbf{X_1}^{(i)}, \mathbf{X_2}^{(i)} \in \mathbb{R}^{2\times n}$ be two n-point curves inputted to the

first and the second arm of the configuration for the ith example of the data with label λ_i. Let $\mathcal{S}_\Theta(\mathbf{X})$ denote the output of the network for a given set of weights Θ for input curve $\mathbf{X}$. The contrastive loss function is given by,

$$\mathcal{L}(\Theta) = \sum_i \lambda_i \, || \, \mathcal{S}_\Theta(\mathbf{X_1}^{(i)}) - \mathcal{S}_\Theta(\mathbf{X_2}^{(i)}) \, || \\ + \, (1 - \lambda_i) \, \max\left(0, \, \mu \, - \, || \, \mathcal{S}_\Theta(\mathbf{X_1}^{(i)}) - \mathcal{S}_\Theta(\mathbf{X_2}^{(i)}) \, ||\right). \tag{4}$$

where μ is a cross validated hyper-parameter, known as margin, which defines the metric threshold beyond which negative examples are penalized. We build a sufficiently large dataset comprising of such positive and negative examples of the transformation from a database of curves. Minimizing the contrastive cost function of the Siamese configuration directs the network architecture to model a function over the curve which is invariant to the prevailing transformation.

Figure 4 demonstrates the results of this exercise for Euclidean transformations. After the training process enumerated above, we input the network with a new unseen curve and visualize the output signatures for various test-cases of euclidean transformations. Ability to handle low signal to noise ratios and efficiency of computation are typical qualities desired in a geometric invariant and hence we also progressively add zero-mean Gaussian noise to the curve to test the robustness of the learned invariant. Importantly, we show a comparison with two axiomatic invariant signatures: differential invariants [10, 9, 38] and integral invariants [25, 28]. Figure 4 shows that the network is able to learn the geometric signature that is not only invariant but also robust to the presence of noise in the data.

3 Learning Isometric Embedding of Manifolds

3.1 Background

Multidimensional scaling (MDS) is a classical algorithm for obtaining the global picture of data using pairwise distances or dissimilarities information. The core idea of MDS is to find an embedding configuration $\mathbf{X} = \left[\mathbf{x_1}, \mathbf{x_2}, \mathbf{x_3} \ldots \mathbf{x_N}\right]$, such that all pairwise distances measured in the embedded space (typically $||\mathbf{x_i} - \mathbf{x_j}||$) are faithful to the given distances $\mathbf{D}_s = \left[d_{ij}^2\right]$ as much as possible. In the context of isometric manifold learning, the MDS framework is enabled by imputing pairwise *geodesic* distances and recovering a low-dimensional embedding output such that pairwise Euclidean distances in this space match the corresponding geodesic distances. Putting simply, this is the action of *flattening* the manifold.

There are two different versions of MDS: *Classical Scaling* and *Least-Squares Scaling*. Classical Scaling is based on the observation that the double centering of a pairwise squared euclidean distance matrix gives an inner-product matrix which can be factored to obtain the desired embedding. Hence, if $\mathbf{H} = \mathbf{I} - \frac{1}{N}\mathbf{1}\mathbf{1}^T$ is the centering

matrix, classical scaling minimizes the strain of the embedding configuration $\mathbf{X}$ and is computed conveniently using the eigen-decomposition of the $N \times N$ matrix $-\frac{1}{2}\mathbf{H}\mathbf{D}_s\mathbf{H}$,

$$\mathbf{X}^*_{CS} = \arg\min_{\mathbf{X}} \; ||\mathbf{X}\mathbf{X}^T + \frac{1}{2}\mathbf{H}\mathbf{D}_s\mathbf{H}||^2_F \tag{5}$$

$$\mathbf{X}^*_{CS} = \mathbf{V}\mathbf{\Lambda}^{\frac{1}{2}}, \text{ where } -\frac{1}{2}\mathbf{H}\mathbf{D}_s\mathbf{H} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{\mathbf{T}}. \tag{6}$$

At the other end, least squares scaling is based on minimizing the misfits between the pairwise distances of $\mathbf{X} = \left[\mathbf{x_1}, \mathbf{x_2}, \mathbf{x_3} \ldots \mathbf{x_N}\right]$ and desired distances $\left[d_{ij}\right]$ measured by the *stress* function

$$\mathbf{X}^*_{LS} = \arg\min_{\mathbf{X}} \; \sigma(\mathbf{X}) \tag{7}$$

$$\sigma(\mathbf{X}) = \sum_{i<j} w_{ij}\left(||\mathbf{x}_i - \mathbf{x}_j|| - d_{ij}\right)^2. \tag{8}$$

Minimization of (8) is typically handled using gradient descent iterations with $w_{ij} = 1$ for all i, j. One particular case was introduced by Leeuw *et al.* [13] under the name SMACOF (Scaling by Majorizing a COmplicated Function).

In practice, the MDS framework is enabled by estimating all pairwise *geodesic* distances with a shortest path algorithm like Dijkstra's [15], and choosing an MDS scaling algorithm to generate low-dimensional embeddings that preserve metric properties of the manifold. Schwartz *et al.* [30, 39] and the Isomap algorithm [36], were the first to suggest populating the inter-geodesic distance matrix $\mathbf{D}_s$ using Dijkstra's algorithm. [18, 40] first suggested the use of consistent approximate of geodesics for the goal of flattening into 2D and [16] employed the Fast Marching method for computing the geodesic distances on 2D surfaces that were then flattened into low dimensional Euclidean spaces like $\mathbb{R}^3$ and $\mathbb{R}^4$ where codimensionality of at least one. They thereby translated the problem of matching non-rigid articulated objects into that of matching rigid ones. See [32] for a recent accurate alternative of matching rigid structures in low dimensional complex spaces $\mathbb{C}^n$.

One of the major drawbacks of the plain MDS approaches is its computational cost and the lack of a principled generalization framework for unseen data. The minimization of objectives 5 and 8 demand computing geodesic distances between *all* pairs of points leading to an expensive eigen-decomposition of a dense $N \times N$ matrix as in the case of classical scaling. This motivated various *fast* MDS algorithms that were based on sampling the manifold and computing the pairwise geodesic distances between these sampled pair of points and using different interpolation philosophies to extract the embedding of the whole dataset [8, 2, 5, 31, 33, 29]. Although these secondary interpolation schemes did provided a speed up, they still lack a principled means to generalize to unseen data. The Landmark Isomap method [14] along with the generalized formulation in [3] showed an out-of-sample extension using a kernel formula based on computing geodesic distances of the new samples to existing landmarks and triangulating their embedding accordingly. In the next

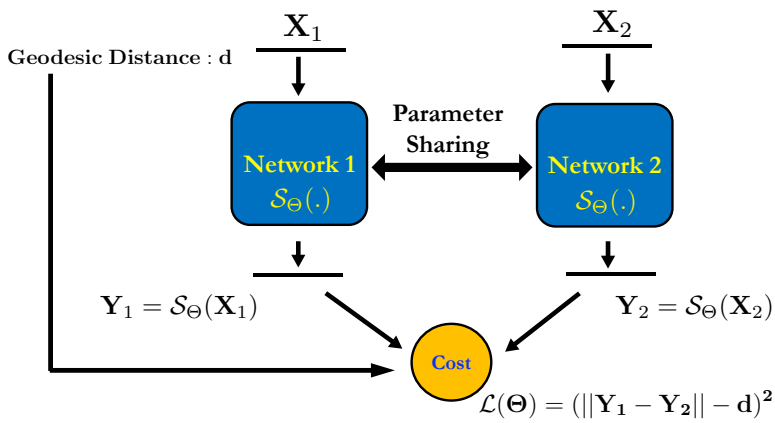

Fig. 5 *Siamese Configuration for MDS:* Each arm of the Siamese network models the map $\mathcal{F}_{\boldsymbol{\Theta}} : \mathbb{R}^M \rightarrow \mathbb{R}^m, \quad m \ll M$. For each training pair $(\mathbf{X}_1, \mathbf{X}_2) \in \mathbb{R}^M$, the loss minimizes the squared difference between the Euclidean distances of the low-dimensional embedding and the geodesic distance d.

subsection we demonstrate a deep-learning approach to perform MDS. This leads to a trivial out-of-sample extension and generalizes to unseen data points. Importantly, the output embedding is robust to noise in comparison to its axiomatic counterparts.

3.2 Deep Multidimensional Scaling

We incorporate the ideas of least squares scaling into the computational infrastructure of the Siamese configuration as shown in Figure 5. For every p^{th} pair of datapoints, we estimate the geodesic distance using a shortest path algorithm and train the network by minimizing the network-parameterized stress function (see Figure 5),

$$\mathcal{L}(\Theta) = \sum_p \left(\, || \, \mathcal{F}_\Theta(\mathbf{X}_1^{(p)}) - \mathcal{F}_\Theta(\mathbf{X}_2^{(p)}) \, || - d^{(p)} \, \right)^2. \tag{9}$$

The expectation here is that with a sufficient number of example pairs, the network learns to model a map $\mathcal{F}_{\boldsymbol{\Theta}} : \mathbb{R}^M \rightarrow \mathbb{R}^m, \quad m \ll M$, from high to low dimension, that preserves isometry.

Instead of minimizing the loss (9) for every pair of points, we sample the manifold using the farthest point sampling strategy [20, 7]. The farthest point sampling strategy, also referred to as the MinMax strategy in [14], is a method for picking landmarks among the points of a discretely sampled manifold such that under certain conditions, these samples cover the manifold as uniformly as possible. Starting from a random selection, the landmarks are chosen one at a time such that each new

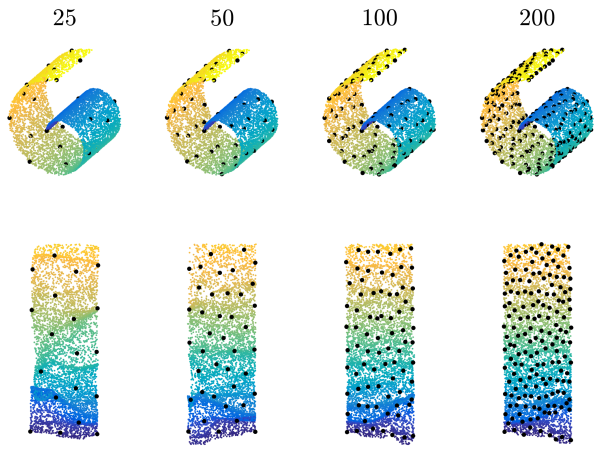

Fig. 6 Learning MDS with Sparse Landmarks: The farthest point sampling algorithm enables a sparse sub-sampling of the manifold, [20]. The network is trained using the loss of equation (9) and the embedding for the entire configuration is visualized. The ability of the network to generate optimal flattening helps gauge the approximation power of the network. See [26]

.

selection from the unused samples has the largest geodesic distance to the set of the selected sample points. We train the network by minimizing the loss in equation 9 by computing the pairwise geodesic distances between K landmarks. Figure 6 provides a visualization of this sampling mechanism along with the output of the network for each case. As expected, we can see that the flattening performance is a function of the number of points sampled on the manifold and this can be used to gauge the approximation power of a particular network architecture in terms of the order of accuracy of the solution [26]. Finally, we tested the robustness of the network to noise in the geodesic distances and compare it with previous sparse MDS methods. The network is shown to perform relatively stable flattening in comparison to an axiomatic approach.

4 Conclusion

In this chapter we summarized metric learning for geometry processing. As compared to a more axiomatic framework of modeling with differential geometry and engineering with numerical analysis, we demonstrated a way of replacing this pipeline with a deep learning framework which combines both these aspects. We demonstrate that the superior representational power of modern learning systems can be leveraged to solve crucial numerical issues involved in the computation of geometric quantities. First, we have demonstrated a method to learn geometric invariants of planar curves.

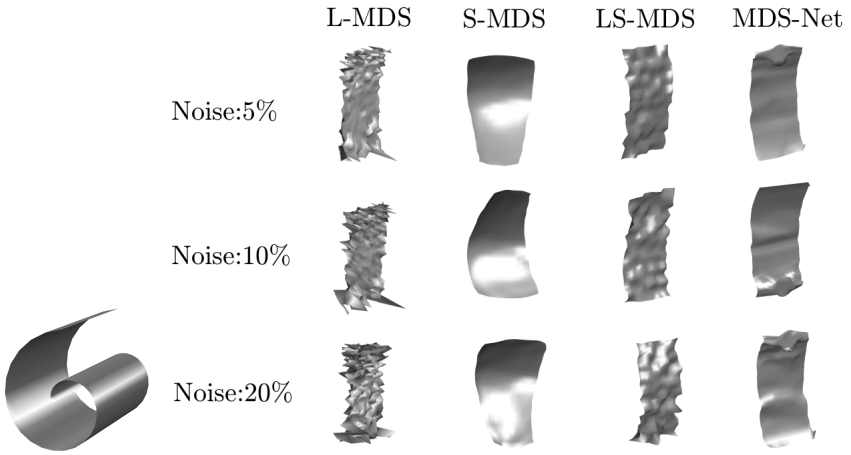

Fig. 7 Stability of the learned isometric embedding to varying levels of noise. The geodesic distance estimates are corrupted with increasing levels of noise and the resultant embedding from different algorithms are visualized. The comparison is shown for principled sparse MDS methods like [34, 4], [2] and [5].

Using just positive and negative examples of euclidean transformations, we show that a convolutional neural network is able to train for an invariant which is numerically more robust as compared to differential and integral invariants. Secondly, we explored an unsupervised deep learning approach to the isometric embedding problem. By training with a few landmarks we see that neural networks integrated into a classical framework like multidimensional scaling demonstrate improved generalization and robustness properties.

References

1. Ackerman, M.: Sophus Lie's 1884 Differential Invariant Paper. Math Sci Press (1976)
2. Aflalo, Y., Kimmel, R.: Spectral multidimensional scaling. Proceedings of the National Academy of Sciences **110**(45), 18052–18057 (2013)
3. Bengio, Y., LeCun, Y.: Scaling learning algorithms towards AI. In: Large Scale Kernel Machines. MIT Press (2007)
4. Bengio, Y., Paiement, J.f., Vincent, P., Delalleau, O., Roux, N.L., Ouimet, M.: Out-of-sample extensions for lle, isomap, mds, eigenmaps, and spectral clustering. In: Advances in neural information processing systems, pp. 177–184 (2004)
5. Boyarski, A., Bronstein, A.M., Bronstein, M.M.: Subspace least squares multidimensional scaling. In: International Conference on Scale Space and Variational Methods in Computer Vision, pp. 681–693. Springer (2017)
6. Bromley, J., Bentz, J.W., Bottou, L., Guyon, I., LeCun, Y., Moore, C., Säckinger, E., Shah, R.: Signature verification using a âĂIJsiameseâĂİ time delay neural network. International

Journal of Pattern Recognition and Artificial Intelligence **7**(04), 669–688 (1993)
7. Bronstein, A.M., Bronstein, M.M., Kimmel, R.: Numerical geometry of non-rigid shapes. Springer Science & Business Media (2008)
8. Bronstein, M.M., Bronstein, A.M., Kimmel, R., Yavneh, I.: Multigrid multidimensional scaling. Numerical linear algebra with applications **13**(2-3), 149–171 (2006)
9. Bruckstein, A.M., Netravali, A.N.: On differential invariants of planar curves and recognizing partially occluded planar shapes. Annals of Mathematics and Artificial Intelligence **13**(3-4), 227–250 (1995)
10. Calabi, E., Olver, P.J., Shakiban, C., Tannenbaum, A., Haker, S.: Differential and numerically invariant signature curves applied to object recognition. International Journal of Computer Vision **26**(2), 107–135 (1998)
11. Cartan, E.: Geometry of Riemannian Spaces: Lie Groups; History, Frontiers and Applications Series, vol. 13. Math Science Press (1983)
12. Chopra, S., Hadsell, R., LeCun, Y.: Learning a similarity metric discriminatively, with application to face verification. In: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), vol. 1, pp. 539–546. IEEE (2005)
13. De Leeuw, J., Mair, P.: Multidimensional scaling using majorization: Smacof in r. Department of Statistics, UCLA (2011)
14. De Silva, V., Tenenbaum, J.B.: Sparse multidimensional scaling using landmark points. Tech. rep. (2004)
15. Dijkstra, E.W.: A note on two problems in connexion with graphs. Numerische mathematik **1**(1), 269–271 (1959)
16. Elad, A., Kimmel, R.: Bending invariant representations for surfaces. In: Computer Vision and Pattern Recognition, 2001. CVPR 2001. Proceedings of the 2001 IEEE Computer Society Conference on, vol. 1, pp. I–I. IEEE (2001)
17. Fidler, T., Grasmair, M., Scherzer, O.: Identifiability and reconstruction of shapes from integral invariants. Inverse Problems and Imaging **2**(3), 341–354 (2008)
18. Grossmann, R., Kiryati, N., Kimmel, R.: Computational surface flattening: a voxel-based approach. IEEE Transactions on Pattern Analysis & Machine Intelligence (4), 433–441 (2002)
19. Hadsell, R., Chopra, S., LeCun, Y.: Dimensionality reduction by learning an invariant mapping. In: 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), vol. 2, pp. 1735–1742. IEEE (2006)
20. Hochbaum, D.S., Shmoys, D.B.: A best possible heuristic for the k-center problem. Mathematics of operations research **10**(2), 180–184 (1985)
21. Hong, B.W., Soatto, S.: Shape matching using multiscale integral invariants. IEEE transactions on pattern analysis and machine intelligence **37**(1), 151–160 (2015)
22. Kendall, D.G.: Shape manifolds, procrustean metrics, and complex projective spaces. Bulletin of the London Mathematical Society **16**(2), 81–121 (1984)
23. Kimmel, R.: Numerical geometry of images: Theory, algorithms, and applications. Springer Science & Business Media (2012)
24. LeCun, Y., Chopra, S., Hadsell, R.: A tutorial on energy-based learning (2006)
25. Manay, S., Hong, B.W., Yezzi, A.J., Soatto, S.: Integral invariant signatures. In: European Conference on Computer Vision, pp. 87–99. Springer (2004)
26. Pai, G., Talmon, R., Bronstein, A., Kimmel, R.: Dimal: Deep isometric manifold learning using sparse geodesic sampling. In: 2019 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 819–828. IEEE (2019)
27. Pai, G., Wetzler, A., Kimmel, R.: Learning invariant representations of planar curves. In: International Conference on Learning Representations (2017)
28. Pottmann, H., Wallner, J., Huang, Q.X., Yang, Y.L.: Integral invariants for robust geometry processing. Computer Aided Geometric Design **26**(1), 37–60 (2009)
29. Rosman, G., Bronstein, A.M., Bronstein, M.M., Sidi, A., Kimmel, R.: Fast multidimensional scaling using vector extrapolation. Tech. rep., Computer Science Department, Technion (2008)
30. Schwartz, E.L., Shaw, A., Wolfson, E.: A numerical solution to the generalized mapmaker's problem: flattening nonconvex polyhedral surfaces. IEEE Transactions on Pattern Analysis and Machine Intelligence **11**(9), 1005–1008 (1989)

31. Shamai, G., Aflalo, Y., Zibulevsky, M., Kimmel, R.: Classical scaling revisited. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 2255–2263 (2015)
32. Shamai, G., Kimmel, R.: Geodesic distance descriptors. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 6410–6418 (2017)
33. Shamai, G., Zibulevsky, M., Kimmel, R.: Accelerating the computation of canonical forms for 3d nonrigid objects using multidimensional scaling. In: Proceedings of the 2015 Eurographics Workshop on 3D Object Retrieval, pp. 71–78. Eurographics Association (2015)
34. Silva, V.D., Tenenbaum, J.B.: Global versus local methods in nonlinear dimensionality reduction. In: Advances in neural information processing systems, pp. 721–728 (2003)
35. Su, B., Su, P.c.: Affine differential geometry. CRC Press (1983)
36. Tenenbaum, J.B., De Silva, V., Langford, J.C.: A global geometric framework for nonlinear dimensionality reduction. science **290**(5500), 2319–2323 (2000)
37. Tombari, F., Salti, S., Di Stefano, L.: Unique signatures of histograms for local surface description. In: European conference on computer vision, pp. 356–369. Springer (2010)
38. Weiss, I.: Noise-resistant invariants of curves. IEEE Transactions on Pattern Analysis and Machine Intelligence **15**(9), 943–948 (1993)
39. Wolfson, E., Schwartz, E.L.: Computing minimal distances on polyhedral surfaces. IEEE Transactions on Pattern Analysis and Machine Intelligence **11**(9), 1001–1005 (1989)
40. Zigelman, G., Kimmel, R., Kiryati, N.: Texture mapping using surface flattening via multidimensional scaling. IEEE Transactions on Visualization and Computer Graphics **8**(2), 198–207 (2002)